AKASH LABORATORY INFORMATION SYSTEM

Commercial Deployment, Networking & Licensing Plan

How to install the LIS once, serve an entire hospital through browsers, connect other sites securely, isolate every buyer's credentials, and turn the current Docker project into a supportable product.

> **Recommended product shape:** one dedicated LIS server/appliance per hospital, browser-only client PCs, HTTPS on the hospital LAN, VPN for other networks, and a signed per-customer licence that never contains your private signing key.

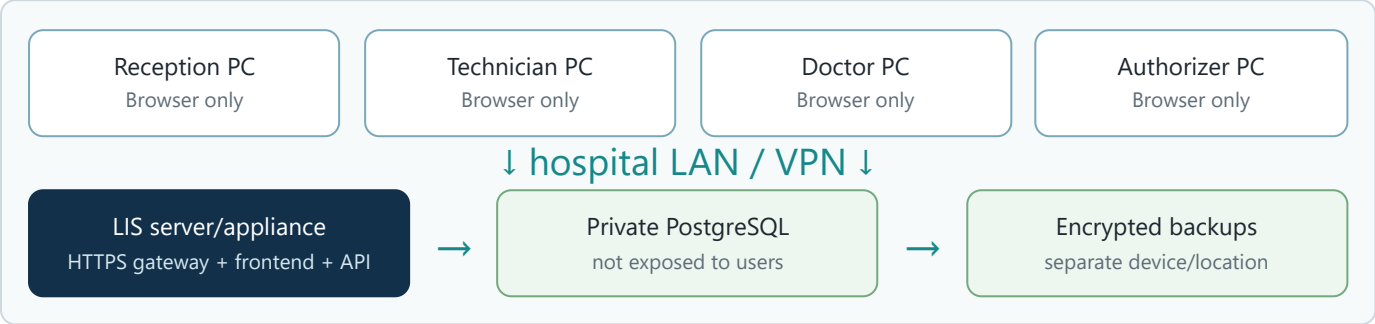
Planning / preview document

Prepared 14 September 2026

EXECUTIVE DECISION

The answer in one page

Do not install Node.js, PostgreSQL, source code, or npm packages on every hospital PC. Install the complete LIS only once on a central server. Every authorised staff member opens one HTTPS address in a browser.



RECOMMENDED

Physical appliance

You supply a configured mini-server with Linux, the container runtime, signed/prebuilt LIS images, firewall, backups and monitoring. The hospital plugs it into the network. This is the closest to "nothing to download."

ALTERNATIVE

Offline installer for hospital server

You supply one package containing prebuilt images and scripts. Hospital IT installs the runtime once on its server. Client PCs still install nothing.

Reality check: the software must run somewhere. You can eliminate installation on every workstation, but the central server still needs an operating system, container runtime, storage and backups. A preconfigured appliance removes almost all buyer-side setup.

What to build next

1. Harden the current Compose deployment: expose only HTTPS, keep API/database private, generate secrets, automate backup/restore.
2. Publish versioned prebuilt frontend/backend images and an offline installation bundle; stop building from source on buyer premises.
3. Add per-installation licensing, session hardening, login throttling, recovery, monitoring, update/rollback and support tooling.
4. Run a clean-machine installation test and a multi-PC hospital acceptance test before selling.

DOCUMENT MAP

What this PDF covers

1	Executive decision	The recommended product shape
2	What “no download” means	Client PC versus central server
3	Same-network architecture	LAN URL, HTTPS, DNS and ports
4	Different-network access	VPN, branch and cloud choices
5	Credentials and licence	Buyer secrets, your keys and machine limits
6	Current LIS readiness audit	What exists and what is missing
7	Required modifications	P0/P1/P2 engineering backlog
8	Commercial package	Online, offline and appliance delivery
9	Operations	Install, backup, update and recovery
10	Hospital member access	Roles, accounts, sessions and devices
11	Security and compliance	Health-data safeguards and Indian context
12	Responsibility matrix	Seller, hospital IT and laboratory
13	Sale-readiness tests	Evidence required before delivery
14	Delivery roadmap	Phases and commercial decisions

Scope: this report is based on the LIS repository at `D:\lis` and its current Docker, authentication, role, report and database design. It does not assess the separate payment-gateway project.

Evidence boundary: this is an architecture/productization assessment based on current files plus the project’s last recorded Docker and browser validations. It is not a new clean-machine installation, penetration test, legal certification or live hospital-network test.

Terms used

Term	Meaning in this plan
Central server / appliance	The one machine that runs the LIS frontend, backend and PostgreSQL containers.
Client PC	A reception, laboratory, doctor or administration workstation; it needs only a modern browser.
Buyer credentials	Hospital-owned user accounts, database secret, integration token and TLS/VPN identity.
Vendor signing key	Your private key used to issue licences or sign releases. It must never be shipped to a buyer.
Single-tenant	One hospital/customer per isolated LIS deployment and database.

PRODUCT MODEL

What “no download on every PC” should mean

The browser is the client. All business logic, clinical data, report generation, secrets and licence verification stay on the central server.

Component	Central LIS server	Hospital member PC
Frontend	Served by Nginx as static files	Opened in browser; nothing installed
Backend API	Runs privately in a container	Never directly configured
PostgreSQL	Private container/managed database	No database client or password
LIS account	Validated server-side	User enters only personal login credentials
Hospital/API secrets	Encrypted/protected server configuration	Never sent to browser
Updates	Applied once to the server	Browser refresh receives the new frontend
Backups	Automated and encrypted	Not performed by staff PCs

Three delivery models

BEST FIRST PRODUCT

Managed on-prem appliance

Highest control, works on LAN if internet fails, simple client experience, supports local analyzer/HMS links. You maintain the server under support contract.

IT-MANAGED

Offline/online server bundle

Lower hardware responsibility for you, but hospital IT must maintain the host, runtime, certificates, storage and backups to your specification.

LATER

Cloud SaaS

Easy updates and multi-site access, but recurring hosting, internet dependency, tenancy/compliance work and secure on-prem analyzer/HMS connectors are required.

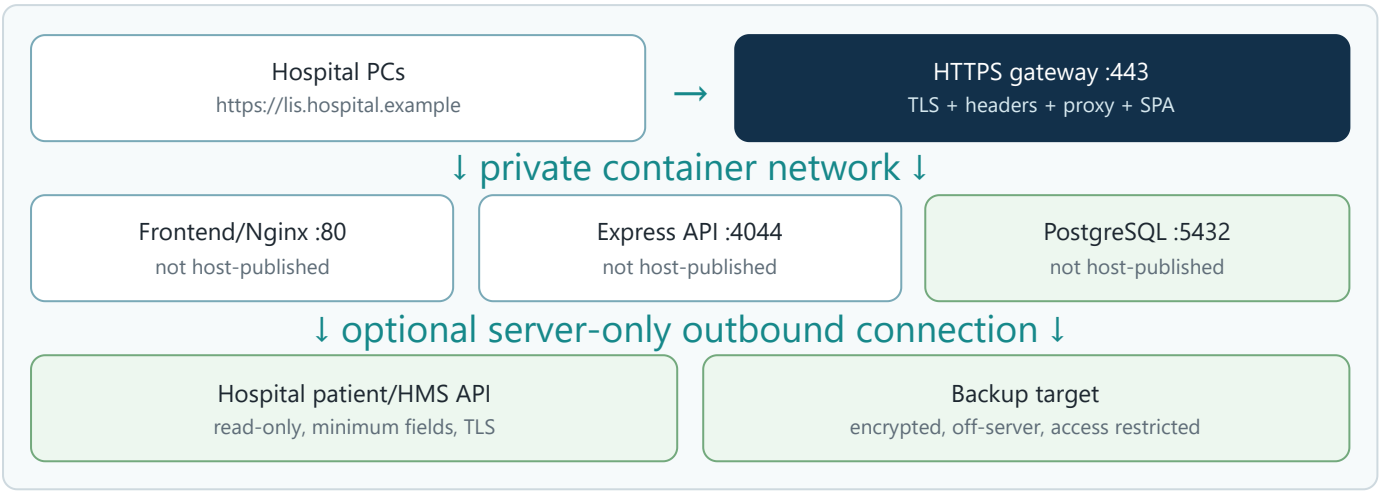
> **Recommendation for this LIS:** start single-tenant and on-premises. Do not add multi-tenant cloud complexity merely to make the first sale. Offer cloud only after tenant isolation, data-location, support and integration requirements are designed and tested.

Suggested server specification for a small hospital

Choose after measuring real workload; a sensible pilot baseline is a business-grade 4-core x86-64 CPU, 16 GB RAM, 512 GB or larger enterprise SSD, TPM, gigabit Ethernet, UPS and a separate backup destination. Use RAID only for availability—it is not a backup. Size storage from report/PDF retention and expected annual order volume.

HOSPITAL LAN

Same-network architecture



Network setup

1. Give the server a reserved/static LAN address, for example `10.20.10.20`.
2. Create hospital DNS such as `lis.hospital.example`. Avoid asking staff to remember an IP address.
3. Use a certificate trusted by hospital PCs. A real domain with DNS validation or a hospital-managed internal certificate authority is preferable.
4. Allow client VLANs to reach only TCP 443 on the LIS server. Deny workstation access to 4044 and 5432.
5. Permit only required outbound destinations from the server: updates, time synchronization, support/VPN, and configured hospital integration.
6. Place analyzers and servers on managed VLANs; document every allowed flow.

> **Do not expose these ports:** the current Compose file publishes PostgreSQL `5432` and backend `4044` on the host. A commercial deployment should publish only the HTTPS entry point. The database and API should remain reachable only inside the private container network, except for tightly controlled administration.

Current quick LAN demonstration versus production

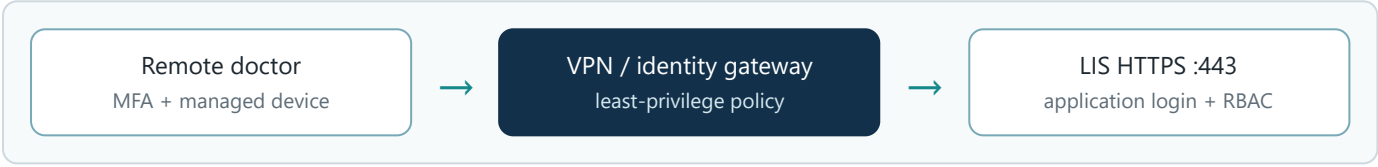
Current	Commercial target
`http://SERVER-IP:8081`	`https://lis.hospital.example` on port 443
HTTP, IP address, explicit port	HTTPS, stable DNS name, trusted certificate
Frontend, backend and DB host ports published	Only gateway 443 published; internal services private

OTHER BUILDINGS AND NETWORKS

Secure access from a different network

A different network must not mean “open the LIS port to the internet.” Add a private, authenticated route to the same HTTPS service.

Situation	Recommended connection	Notes
Second building / branch LAN	Site-to-site VPN between firewalls	Staff keep using the same internal URL. Restrict routes and ports to the LIS service.
Individual doctor from home	User VPN / zero-trust access with SSO and MFA	Authorize both identity and device where possible. Still require the LIS login.
Hospital has no VPN team	Managed overlay VPN such as Tailscale, with grants/ACLs and device approval	Use organisation-owned identities; tag the server, not user devices. Review vendor terms and data path with hospital IT.
Public cloud LIS	Public HTTPS frontend/API behind WAF; private database; secure on-prem connector	Requires much more work: tenancy, compliance, monitoring, regional hosting, data-processing contracts and disaster recovery.



Rules that remain true remotely

- Network membership is not sufficient authorization. Every person uses a unique LIS account and least-privilege role.
- Never forward PostgreSQL `5432`, API `4044`, Docker socket or SSH/RDP broadly to the public internet.
- Remote support should be time-limited, approved, logged and removable by hospital IT.
- Define idle logout, lost-device removal, access review and emergency revocation procedures.
- If connectivity is unreliable, decide exactly which workflows remain available. Do not allow stale or draft reports to appear released.

> **Internet outage:** an on-prem LAN deployment can continue working locally, but cloud identity, remote sites and external patient APIs may not. Document downtime registration, result entry, authorization, report issue and later reconciliation before go-live.

Tailscale is an example, not a mandatory dependency. A hospital-managed IPsec/WireGuard solution may be more appropriate. NIST zero-trust guidance says network location alone should not create implicit trust; identity and resource authorization still matter.

How to isolate buyer credentials and limit installations

Separate credentials by owner

Secret / identity	Owner	Where it should live
LIS staff passwords	Hospital	Hashed in the buyer's database; unique account per person
PostgreSQL password and JWT/session keys	Buyer installation	Generated during installation; server secret store/root-only file; never in image/source
HMS/patient API token	Hospital	Central server only; read-only/minimum scope; never sent to browsers
TLS/VPN keys	Hospital or managed-service account	Gateway/OS secret store with rotation and revocation
Release-signing and licence-signing private keys	You, the vendor	Offline/HSM/CI secret; never copied to buyer server
Licence file	Issued by vendor to buyer	Buyer server; contains signed claims and vendor public-key verification only

> **Never sell a copy containing your current `.env`.** The safe package contains only a template. Generate fresh database, JWT, encryption and support credentials for every hospital. If a real secret was ever included in a shared archive, rotate it.

Recommended licence mechanism

1. **The server creates an installation key pair.** Prefer TPM-backed/non-exportable storage when available. The public key becomes the stable installation identity.
2. **You issue a signed licence document.** Include customer ID, installation public-key hash, edition, enabled modules, issue/expiry dates and optional maximum active users/sites.
3. **The LIS verifies with an embedded public key.** Your private signing key is never present, so the buyer cannot create another valid licence by copying files.
4. **Support safe re-hosting.** Hardware fails. Provide an auditable deactivate/reissue process rather than binding only to MAC address or disk serial.
5. **Fail safely.** Expired licensing may block new non-critical work after warnings/grace, but must not destroy data. Keep read-only access, export, backup and recovery available under the contract.

Online versus offline activation

Online entitlement	Central licence service checks periodically; easier revocation and usage limits; requires reliable internet and a documented grace period.
Offline signed licence	Best for isolated hospitals; licence is generated from an activation request and imported once; renewal/re-hosting is manual.
Hybrid	Recommended commercial default: online where available, signed offline lease with 30–90 day grace for outages.

No on-premises DRM is impossible to bypass by a buyer who has full administrator and hardware access. Containers protect delivery convenience, not source secrecy. Use technical controls together with licence/EULA terms, signed releases, support value and customer-specific credentials.

CURRENT REPOSITORY REVIEW

What the LIS already has—and what blocks a sale

Useful foundations already present

Status	Current capability	Commercial value
PRESENT	Docker Compose with frontend, backend and PostgreSQL; persistent database volume and health checks	Good base for a one-server appliance
PRESENT	Nginx serves the React SPA and proxies `/api`; frontend uses relative `/api`	One browser URL and same-origin API are already natural
PRESENT	Roles: ADMIN, IT, DOCTOR, TECHNICIAN, AUTHORIZER; server-side permissions and active-user reload	Hospital members can receive role-appropriate access; account disablement takes effect on next request
PRESENT	No default staff account; first administrator is created explicitly; bcrypt password hashing	Avoids a universal shipped password
PRESENT	Audit events, protected authorized results, append-only clinical notes and authenticated PDF reports	Important clinical accountability controls
VALIDATED	Latest recorded production Docker smoke covered migration/startup, DB health, admin bootstrap, doctor login, authorized-result access, notes and PDF generation	Strong implementation evidence, though not a buyer-machine test

Sale blockers / gaps

Status	Observed gap	Required outcome
P0	Compose builds source locally and Dockerfiles run `npm ci`; buyer needs source/build network and toolchain	Prebuilt versioned images plus offline image bundle or preconfigured appliance
P0	HTTP only; frontend 8081, backend 4044 and PostgreSQL 5432 are host-published	Trusted HTTPS gateway on 443; API/database private
P0	Development credentials are present in `.env.example`; no installer-generated secret flow	Per-hospital random secrets, protected permissions, rotation and redacted diagnostics
P0	Manual backup command; no retention, encryption, off-server copy, monitoring or scheduled restore drill	Automated, encrypted, tested backup/restore with documented RPO/RTO
P1	12-hour bearer JWT stored in browser localStorage; no visible server session registry or login rate limiter	Short/rotating sessions, safer cookie/token design, logout/revoke, throttling and security events
P1	No licence/activation, signed-release update, rollback or buyer entitlement system	Signed per-install licence and versioned update channel
P1	No clean-machine installer/uninstaller, preflight, support bundle or disaster-recovery automation	Repeatable delivery and support without developer intervention
P2	Single deployment branding/config; no formal tenant/site model	Keep one hospital per deployment initially; add multi-site/tenant only if commercially required

Packaging mismatch to resolve: the repository contains an untracked backend image archive named `lis-backend-1.1.0.tar`, while Compose tags the backend build as `1.0.0`, and no matching frontend/database offline bundle was observed. Treat this as a development artifact, not a release package.

ENGINEERING BACKLOG

Exact modifications recommended

P0 Required before the first commercial hospital

1. **Production Compose:** replace `build:` with pinned image references/digests; add a dedicated internal network; remove host mappings for backend/database; publish only gateway 443 (and optionally redirect 80→443).
2. **TLS gateway:** add Caddy/Nginx/Traefik configuration, certificate installation/renewal, HSTS after HTTPS is stable, request-size/timeouts, security headers and real client-IP handling from trusted proxies.
3. **Installer:** preflight CPU/RAM/disk/ports/time/DNS, create directories and random secrets, import/pull images, start services, wait for health, display the URL and write a non-secret installation report.
4. **Secrets:** remove usable defaults; protect environment/secret files from normal users; never bake secrets into images; add safe rotation procedures for DB, JWT/session and integrations.
5. **Backup/restore:** scheduled `pg_dump` or managed equivalent, encryption, checksums, retention, off-server copy, failure alerts, restore into a disposable instance and quarterly recovery exercise.
6. **Release process:** immutable semantic versions, database migration compatibility, SBOM/dependency scan, signed checksums/images, update notes and tested rollback.
7. **Clinical acceptance:** laboratory-authorized catalogue/ranges/report layout, analyzer validation per model, printer/barcode checks, audit review and documented downtime process.

P1 Security and support hardening

1. Add login throttling, failed-login/security audit, password reset by authorised admin, forced password change, password policy and optional MFA/SSO.
2. Replace long-lived localStorage-only auth with a threat-modelled session design: short access lifetime plus rotating refresh/session records, or secure HttpOnly/SameSite cookies with CSRF protection.
3. Add active-session listing/revocation, idle timeout, absolute timeout, signing-key rotation and emergency logout of all users.
4. Add a licence module that verifies signed entitlements locally, exposes only non-sensitive status, supports grace/re-hosting and audits activation changes.
5. Add structured logs, disk/backup/database/certificate health, alerting, redacted support bundle and time synchronization checks.
6. Add upgrade/rollback tooling that takes a verified pre-upgrade backup and prevents incompatible downgrades after irreversible migrations.

P2 Scale after real buyer demand

- High availability, warm standby and point-in-time recovery for hospitals whose downtime requirements justify the cost.
- Central update/licence portal, fleet inventory and remote support broker with hospital-controlled approval.
- True multi-site or multi-tenant design, only after defining data ownership, identifiers, reporting and tenant isolation tests.
- Enterprise SSO (OIDC/SAML), SCIM provisioning, device posture and fine-grained contextual authorization.

DELIVERY ARTIFACTS

What you hand to a buyer

Preferred: preconfigured appliance

- Hardened supported OS, auto-starting container service, firewall and automatic security updates under a maintenance policy.
- Pinned LIS frontend/backend/PostgreSQL images already loaded; no source tree, npm install or compiler on-site.
- Hospital-specific configuration and licence installed during commissioning—not copied from your development PC.
- UPS connection, recovery media/instructions, asset label, server DNS/IP worksheet and sealed recovery credentials handed to authorised hospital IT.

Alternative: one offline server bundle

Example release layout

```
Akash-LIS-1.2.0/
  install.ps1                # signed, idempotent installer
  uninstall.ps1              # preserves/exports data by default
  compose.production.yml     # image references, no source builds
  images/
    lis-frontend-1.2.0.tar
    lis-backend-1.2.0.tar
    postgres-16.x.tar
  config/
    lis.env.template         # no passwords or real tokens
    gateway.conf.template
  checksums.sha256
  checksums.sha256.sig
  docs/
    buyer-install.pdf
    admin-manual.pdf
    backup-restore.pdf
    release-notes.pdf
```

Container images can be saved to archives and loaded on an offline host. The installer verifies signature/checksums before loading, generates secrets, starts the pinned release and performs health checks.

Online package

Use a private registry with short-lived, buyer-specific read credentials. The buyer pulls signed versioned images; its token must not grant push or access to other customers. Pin production to immutable digest/version rather than `latest`.

What must not be in the package

- Your development `.env`, vendor private keys, registry admin token, personal VPN account or another hospital's data/configuration.
- Real sample patients, database dumps, logs or PDFs from development/other hospitals.
- A universal administrator password or hidden vendor backdoor.
- Unlicensed third-party fonts, libraries, OS images or database components. Maintain an SBOM and notices.

> **Source protection:** shipping compiled/containerized code raises the copying effort but is not absolute protection. Your enforceable protection is a signed licence, customer contract, release signatures, confidential vendor keys and ongoing update/support value.

LIFECYCLE

Install once; operate for years

Commissioning flow

1. **Site survey:** server ownership, VLANs, DNS, certificate, UPS, backup target, printers, barcode devices, analyzers, HMS API, expected users/volume and downtime targets.
2. **Install:** verify signed release, generate secrets, load images, start containers, run idempotent migrations and wait for all health checks.
3. **Activate:** import signed hospital licence; record customer/deployment ID without embedding patient data in your licence service.
4. **Configure:** hospital identity, authorised catalogue/ranges, departments, staff roles, printers and optional read-only patient lookup.
5. **Validate:** synthetic patient/order/result/authorization/amendment/report; permissions; multi-PC access; backup and restore; outage procedure.
6. **Handover:** URLs, recovery ownership, backup status, support boundaries, training, acceptance record and signed data-responsibility schedule.

Backup policy to agree contractually

Decision	Example baseline
RPO: acceptable data loss	24 hours for daily dump; reduce with more frequent dumps/PITR if required
RTO: acceptable recovery time	4–8 hours for small single-server deployment; prove it with a drill
Copies	At least production + encrypted local backup + encrypted off-server/off-site copy
Retention	Daily/weekly/monthly schedule aligned to hospital policy and applicable law
Verification	Checksum each backup, alert failures, restore quarterly to an isolated test system

Update sequence

1. Read release notes and confirm supported upgrade path.
2. Run preflight and take a verified backup.
3. Pull/load signed images into a staging or maintenance window.
4. Apply migrations, start, run health and workflow smoke tests.
5. Rollback application version if safe; otherwise restore according to the migration runbook.
6. Record version, operator, time, checks and outcome.

> **Never treat “container is running” as proof.** Verify database-connected health, login, patient registration/search, order creation, result entry, authorization, report PDF, deep-link reload and backup restore.

Hospital member access

The current LIS already has a strong role starting point. Keep role enforcement on the backend; UI hiding is only convenience.

Role	Typical access	Commercial control to confirm
ADMIN	Laboratory configuration, staff/access, clinical workflow	At least two named administrators; sensitive actions re-authenticated and audited
IT	Clinical staff/departments only in the current design	No automatic right to see results unless explicitly required and approved
DOCTOR	Hospital-wide status, authorised results, print/download, clinical notes	Draft values hidden; remote access uses MFA/VPN; report access audited
TECHNICIAN	Registration/result workflow	Cannot authorize own results unless policy explicitly permits a separate role
AUTHORIZER	Review, authorize, view and issue released reports	Qualification and signature governance set by laboratory leadership

Account rules

- One named account per person—never “lab/lab123” shared by an entire shift.
- Hospital admin/IT creates, changes and disables accounts; no public signup after bootstrap.
- Review active users and role/department membership at least quarterly and immediately after transfer/exit.
- Lock or throttle repeated failed login attempts; support controlled unlock/reset without exposing old passwords.
- Require re-authentication for highly sensitive actions if the clinical policy requires it.
- Use synchronized time so authorization, clinical-note and audit timestamps are defensible.

Device access choices

LAN only	Firewall allows hospital workstation VLANs to HTTPS 443. Simple, but anyone on those VLANs can reach the login page.
Managed-device certificate	Mutual TLS/device certificate adds a machine gate before LIS login; more administration and recovery work.
VPN/device posture	Best for remote access; identity, MFA and approved-device checks happen before reaching the LIS.

> **Do not confuse licence with staff permission.** A licence authorizes one hospital deployment to run the product. A user account/RBAC decision authorizes a particular person to see or change clinical information. Both checks are needed and must remain separate.

SECURITY AND COMPLIANCE

Minimum health-data safeguards

Confidentiality

- HTTPS everywhere
- Least-privilege roles
- Private DB/API networks
- Encrypted backups
- Server disk encryption with recovery-key governance

Integrity

- Immutable/versioned releases
- Server-side authorization
- Audit trails
- Authorized report revisions
- Signed updates and checksums

Availability

- UPS and disk monitoring
- Automated backups
- Restore drills
- Documented downtime process
- Supported OS/database versions

Accountability

- Named users
- Login/security events
- Access review
- Time synchronization
- Incident and breach process

India-specific planning context

- Digital personal and health information requires a documented lawful purpose, data-minimization, security, retention/deletion and data-subject response process. India's DPDP Act/Rules have phased commencement; obtain current Indian legal review before contracting or go-live.
- If integrating with ABDM/ABHA, follow the applicable ABDM Health Data Management Policy and current technical/consent specifications. A local LIS deployment is not automatically ABDM-compliant merely because it has logins and audit logs.
- CERT-In's 2026 information-security guidance includes logging and retention expectations; translate applicable requirements into the hospital security policy and support contract.
- Decide who is the data fiduciary/controller, who is processor/service provider, where data/backups reside, who may access support data, breach notification duties, retention, exit/export and secure deletion.

> **Not a certification:** encryption, Docker, RBAC or this plan alone does not certify legal compliance. Compliance depends on the product, configuration, hospital process, contracts, staff behaviour and evidence over time.

Security testing before sale

- Threat model, dependency/SBOM scan, secrets scan, static analysis and API authorization matrix tests.
- Independent web/API penetration test focused on authentication, broken access control, report/PDF access, injection, file paths, CORS, headers and rate limits.
- Container/image vulnerability review and patch policy with a supported response time.
- Restore test using encrypted production-like backup and loss-of-server exercise.

CONTRACT AND HANDOVER

Who is responsible for what

Area	You / LIS vendor	Hospital / buyer
Software	Secure releases, migrations, known-issue disclosure, support and update policy	Use supported version; approve maintenance windows
Server	Specified appliance/build, hardening guide and health checks	Physical security, power/UPS, network and authorised admins
Clinical configuration	Tools and auditability	Qualified approval of tests, units, methods, ranges, calculations and report wording
Accounts	RBAC design, secure reset/revoke capability	Create named users, review roles, disable leavers and train staff
Backups	Automation, alerting and restore procedure if contracted	Approve retention/location, protect keys, review failures and participate in drills
HMS/analyzers	Documented read-only/API adapters and validation tools	Vendor specifications, restricted credentials, firewall access and clinical validation
Privacy/security	Product safeguards, vulnerability/incident channel, processor obligations	Lawful use, notices/consents where required, local policies and access governance
Exit	Documented data export, licence deactivation/re-host and deletion evidence	Authorise recipient, retain records and revoke access

Commercial decisions to put in writing

- Perpetual licence plus annual maintenance, or subscription; modules, user/site limits and renewal grace.
- Who owns hardware, domain, certificates, VPN organisation, backup target and recovery keys.
- Support hours, severity definitions, response targets, remote-access approval and excluded third-party failures.
- Supported browsers, OS/runtime/database versions and end-of-support notice.
- Data ownership, confidentiality, support-log redaction, breach cooperation and subcontractors.
- Upgrade entitlement, rollback limits, custom integration charges and clinical-change approval.
- Export/termination assistance, deletion process and re-hosting due to hardware failure.

> **Good sales promise:** "The LIS is installed once on a managed server. Hospital staff use a browser from approved networks. Each hospital owns its users and operational data. Vendor secrets and licences are separated per customer. Updates and backups follow a documented, testable process."

PROOF BEFORE DELIVERY

Sale-readiness acceptance checklist

Clean server install

- ☐ Fresh supported machine, no Node/npm/source
- ☐ Offline bundle installs without internet, or online pull uses read-only buyer token
- ☐ Unique secrets generated; placeholders rejected
- ☐ Services auto-start after reboot
- ☐ Only intended host ports are listening
- ☐ Health check and non-secret support report pass

Network and browser

- ☐ Trusted HTTPS DNS URL opens from at least three hospital PCs
- ☐ Deep link and browser reload work
- ☐ Backend/database cannot be reached from staff VLAN
- ☐ Remote VPN access works only for approved identity/device
- ☐ Revoked remote user/device loses access

Licence

- ☐ Valid licence activates correct customer/features
- ☐ Copied licence fails on unlicensed installation
- ☐ Tampered licence fails cleanly and is audited
- ☐ Offline/grace and hardware re-host paths tested
- ☐ Expiry never deletes or corrupts clinical data

Roles and clinical workflow

- ☐ ADMIN/IT/DOCTOR/TECHNICIAN/AUTHORIZER matrix tested server-side
- ☐ Disabled or changed user loses old permissions promptly
- ☐ Draft results never appear to doctors
- ☐ Amendment blocks re-release until authorization
- ☐ Report PDF and print show correct revision/notes
- ☐ Audit events identify actor and synchronized time

Data and recovery

- ☐ Automated encrypted backup completes
- ☐ Backup is copied off the server
- ☐ Restore succeeds to a clean isolated instance
- ☐ Restored records/reports match checks
- ☐ Disk-full and backup-failure alerts reach an owner
- ☐ Server-loss and downtime procedure rehearsed

Upgrade

- ☐ Signed update accepted; tampered update rejected
- ☐ Pre-upgrade backup verified
- ☐ Migration/startup and workflow smoke pass
- ☐ Supported rollback/restore path demonstrated
- ☐ Installed version recorded in admin/health output

Do not call the product commercially ready until the clean-machine and restore tests pass. A successful build, Compose configuration or developer-machine Docker run does not prove buyer installation, recovery or hospital-network security.

RECOMMENDED EXECUTION

Practical roadmap from current LIS to product

Phase	Deliverable	Exit evidence
1. Production network baseline	Private Compose networks, only HTTPS gateway exposed, generated secrets, production env schema, LAN DNS/certificate guide	Port scan, HTTPS test, reboot test, multi-PC LAN smoke
2. Repeatable package	Versioned prebuilt images, signed checksums, online/offline installer, no source/npm on buyer server	Clean offline machine installs and runs from one bundle
3. Operations	Scheduled encrypted backup, restore, monitoring, support bundle, upgrade/rollback	Disaster-recovery and upgrade rehearsal
4. Security hardening	Session redesign, login throttling, reset/revoke, security audit, dependency/image review	Authorization regression plus independent penetration test
5. Licensing	Installation identity, signed entitlements, grace/re-host, admin status and contract	Copy/tamper/expiry/recovery tests
6. Pilot hospital	Site survey, training, clinical configuration, network/VPN, acceptance and support runbook	Signed pilot acceptance and monitored support period

Decisions needed before engineering starts

1. **Delivery:** vendor-supplied appliance or hospital-owned server bundle?
2. **Connectivity:** LAN only, branch-to-branch VPN, named remote users, or future cloud?
3. **Licence:** perpetual + maintenance or subscription; offline grace; user/site/module limits?
4. **Availability:** required RPO/RTO and whether a warm standby is commercially justified?
5. **Support:** who controls VPN, backup keys, certificates, OS patching and incident response?

> **Recommended first implementation:** managed on-prem appliance, single hospital per server, LAN HTTPS, optional named-user VPN, offline-capable signed licence, automated daily encrypted backup, and a tested one-command update/rollback process.

What not to build first

Do not start with multi-tenant cloud hosting, complex per-device staff licensing, broad internet exposure or analyzer integrations without exact vendor protocols. First make the existing single-hospital workflow installable, secure, recoverable and supportable.

SOURCES AND EVIDENCE

References

Repository evidence inspected

- `D:\lis\docker-compose.yml` — services, published ports, volume, build-based delivery and health checks.
- `D:\lis\backend\Dockerfile` and `frontend\Dockerfile` — on-site image build currently runs npm dependency installation.
- `D:\lis\docker\nginx.conf` — HTTP SPA/API gateway and internal backend proxy.
- `D:\lis\.env.example` and backend environment validation — current configuration/placeholder handling.
- Authentication, permission and frontend API code — bcrypt, JWT lifetime/storage, database user reload and role permissions.
- `D:\lis\docs\doctor-access.md` — doctor access, staff administration, notes, reports and latest documented validation.

External primary/official guidance

1. Docker Docs — docker image load and docker image save example (offline image archives).
2. PostgreSQL — Backup and Restore and SQL Dump.
3. NIST SP 800-207 — Zero Trust Architecture.
4. Tailscale — Access control and site-to-site networking (one possible managed VPN approach).
5. MeitY — Digital Personal Data Protection Act, 2023, DPDP Rules, 2025 and official commencement notification.
6. ABDM/NDHM Health Data Management Policy.
7. CERT-In — Guidelines on Information Security Practices (2026).
8. CISA — eliminating default passwords / secure-by-design guidance.

> **Professional review:** use qualified Indian legal/privacy counsel, a security assessor and the hospital's laboratory leadership before production. This document is a technical productization plan, not legal advice or clinical certification.

Final recommendation

Package the LIS as a single-hospital server appliance. Make the secure path the easiest path: plug in the server, assign DNS/certificate, create named users, verify backups, and let staff work from browsers. Add remote access only through identity-controlled VPN, and keep licensing separate from clinical authorization and data ownership.